

I tipi Valore ed i tipi Riferimento.

1. Tipi di valore

Un **tipo di valore** è un tipo di dato che contiene direttamente i dati nella propria locazione di memoria. Questo implica che, quando si dichiara una locazione di un tipo valore (per es. si dichiara una variabile intera) la si può usare immediatamente, poiché il suo spazio dei dati è già pronto.

Caratteristiche dei tipi di valore:

- Ogni variabile di tipo valore è indipendente. Modificare una variabile non influisce su un'altra, anche se sono uguali.
- Quando una variabile di tipo valore viene assegnata a un'altra, viene creata una **copia** del valore.
- Non dipendono dalla memoria heap; sono allocati nello **stack**.

Esempi di tipi di valore in C#:

- `int` (intero)
- `float` (numero con virgola mobile)
- `double` (numero con virgola mobile a doppia precisione)
- `char` (carattere)
- `bool` (valore booleano)

2. Tipi di riferimento

Un **tipo di riferimento** è un tipo di dato che **non** memorizza direttamente il suo valore, ma un **riferimento alla memoria** dove il valore è effettivamente memorizzato.

Quando si dichiara una variabile di tipo riferimento si dispone del solo spazio che contiene l'eventuale indirizzo che inizialmente è un indirizzo nullo. Per questo il tentativo di usarlo solleva un errore:

```
int [] v ;
```

```
v [0] = 17; //Errore!
```

Per poter usare la locazione di tipo riferimento è necessario prima allocare la memoria, ovvero farne richiesta al sistema operativo che trova la memoria, la prepara per la locazione ed infine aggancia il riferimento alla variabile.

Caratteristiche dei tipi di riferimento:

- Le variabili di tipo riferimento **non sono indipendenti**. Se una variabile viene modificata, anche la copia (perché punta alla stessa area di memoria) risente del cambiamento.
- Quando una variabile di tipo riferimento viene assegnata a un'altra, entrambe le variabili puntano allo stesso oggetto in memoria.
- Sono allocati nella **heap** (memoria dinamica).

Esempi di tipi di riferimento in C#:

- `string`
- Classi e oggetti (definiti dall'utente)
- `array`

ATTENZIONE: Esempio di comportamento delle stringhe

In C#, le stringhe sono tipi di riferimento, ma sono trattate come **immutabili**. Questo significa che, anche se una stringa è un tipo di riferimento (e normalmente i tipi di riferimento possono essere modificati tramite il loro riferimento), una stringa non può essere modificata direttamente dopo che è stata creata.

Ad esempio, quando si cambia il valore di una stringa, si sta effettivamente creando una nuova stringa in memoria. La stringa originale rimane invariata.

```
string str1 = "Ciao";
```

```
string str2 = str1; // str2 è una copia di str1, ma entrambe puntano allo stesso valore
```

```
str1 = "Hello"; // Qui str1 viene "modificata", ma in realtà viene creata una nuova stringa
```

```
Console.WriteLine(str1); // Stampa "Hello"
```

```
Console.WriteLine(str2); // Stampa "Ciao"
```

Come faccio a modificare una stringa?

Attraverso dei **metodi** che creano nuove stringhe. Ad esempio, metodi come `Replace`, `Substring`, `ToLower`, `ToUpper`, ecc., creano una nuova stringa con il risultato dell'operazione.

Modalità di passaggio dei valori a una funzione

Quando si passa una variabile a una funzione, esistono due modalità principali: **passaggio per valore** e **passaggio per riferimento**. La differenza principale sta nel fatto che la funzione può o meno modificare la variabile originale, in base alla modalità di passaggio.

a) Passaggio per valore

Quando una variabile viene passata **per valore** a una funzione, **una copia del valore** viene passata alla funzione. La funzione lavora sulla copia e non può modificare la variabile originale.

Esempio di passaggio per valore (C#):

```
void Modifica(int x)
{
    x = 10; // Modifica la copia di x, non la variabile originale
}

int numero = 5;
Modifica(numero);
Console.WriteLine(numero); // Stampa 5, la variabile numero non è stata
modificata
```

In questo caso, la variabile `numero` non viene modificata, perché la funzione `Modifica` riceve una **copia** di `numero`, non il valore stesso. Modificare la copia non influisce sulla variabile originale.

b) Passaggio per riferimento

Quando una variabile viene passata **per riferimento**, la funzione riceve **un riferimento alla variabile originale** e può modificarla direttamente.

In C#, per passare un parametro per riferimento si utilizza la parola chiave `ref` o `out`.

Esempio di passaggio per riferimento con `ref` (C#):

```
void Modifica(ref int x)
{
    x = 10; // Modifica la variabile originale
}

int numero = 5;
Modifica(ref numero);
Console.WriteLine(numero); // Stampa 10, la variabile numero è stata modificata
```

In questo esempio, `numero` viene passato alla funzione `Modifica` tramite il **riferimento**, quindi la funzione modifica direttamente la variabile originale. È importante notare che la variabile deve essere inizializzata prima di essere passata a una funzione che utilizza `ref`.

Esempio di passaggio per riferimento con `out` (C#):

La parola chiave `out` è simile a `ref`, ma con una differenza: la variabile passata con `out` non deve essere inizializzata prima di essere passata alla funzione. La funzione è obbligata a inizializzare la variabile.

```
void ImpostaValore(out int x)
{
```

```

    x = 10; // La variabile x è inizializzata all'interno della funzione
}

int numero;
ImpostaValore(out numero);
Console.WriteLine(numero);    // Stampa 10, la variabile numero è stata
                              // inizializzata dalla funzione

```

In questo esempio, `numero` non è inizializzato prima della chiamata a `ImpostaValore`, ma viene automaticamente inizializzato all'interno della funzione.

4. Passaggio di tipi di riferimento

Quando si passa un tipo di riferimento a una funzione, tecnicamente si sta passando un **riferimento all'oggetto**. Questo significa che la funzione può modificare l'oggetto a cui il riferimento punta, ma non può cambiare il riferimento stesso (a meno che non venga passato esplicitamente con `ref`).

Esempio di passaggio di un tipo di riferimento (C#):

```

class Persona
{
    public string Nome { get; set; }
}

void ModificaPersona(Persona p)
{
    p.Nome = "Mario"; // Modifica l'oggetto
}

Persona persona = new Persona();
persona.Nome = "Giovanni";
ModificaPersona(persona);
Console.WriteLine(persona.Nome); // Stampa "Mario", l'oggetto è stato modificato

```

In questo esempio, la funzione `ModificaPersona` modifica l'oggetto `persona` passato come parametro, ma non modifica il riferimento alla variabile `persona`. In altre parole, **l'oggetto** a cui `persona` punta viene modificato, ma non il riferimento stesso.